FIAP – Faculdade de Informática e Administração Paulista

Análise e Desenvolvimento de Sistemas

EcoRank

API Restful de ranking de ações sustentáveis

Domain Driven Design using Java – Entrega Final

Nome da solução: EcoRank

Nome da equipe: SoulUp

Integrantes

[Nome completo do integrante 1] – RM570464

[Nome completo do integrante 2] – RM [número]

[Nome completo do integrante 3] – RM [número]

Professor: Eduardo Gondo

São Paulo – 2026

Sumário

1. Objetivo e escopo do projeto
2. Breve descrição das principais funcionalidades
3. Arquitetura e tecnologias utilizadas
4. Tabela de endpoints (API Restful)
5. Exemplos de requisição e resposta
6. Protótipo – prints das telas implementadas
7. Modelo de Entidade-Relacionamento (MER)
8. Diagrama de classes atualizado
9. Procedimentos para rodar a aplicação
10. Deploy da aplicação na nuvem (Render)
11. Links da entrega

1. Objetivo e escopo do projeto

O EcoRank é uma solução que incentiva hábitos sustentáveis por meio de gamificação. O usuário registra ações sustentáveis (reciclagem, plantio, economia de água e de energia, entre outras), anexa um link de evidência (foto ou vídeo) e, quando a ação é aprovada, ganha pontos, sobe de nível, avança em missões, desbloqueia conquistas e disputa posições em um ranking.

O objetivo desta entrega é transformar a aplicação de console desenvolvida anteriormente em uma API Restful com Spring Boot, pronta para ser consumida por um front-end e publicada na nuvem. A API segue a arquitetura em três camadas vista em aula (Resource/Controller, Service e Repository), usa JDBC com DataSource e banco de dados Oracle.

Escopo:

- Cadastro e manutenção (CRUD) de usuários, ações sustentáveis, missões, conquistas e níveis;
- Cálculo de pontos, validação de evidências, atualização de nível e progresso de missões;
- Ranking de usuários ordenado por pontuação;
- Liberação de CORS para o front-end e deploy via Docker no Render.

Fora do escopo: autenticação/login de usuários e o desenvolvimento do front-end.

2. Breve descrição das principais funcionalidades

- **Usuários:** cadastro com nome e email (único). Todo usuário começa com 0 pontos e nível Bronze.
- **Ações sustentáveis:** o usuário informa categoria, descrição e link de evidência. O sistema calcula os pontos pela categoria (Reciclagem 50, Economia de Agua 40, Plantio 70, Economia de Energia 60, demais 20) e valida a evidência (link com “http” ou terminado em .jpg, .png ou .mp4). Evidência válida deixa a ação APROVADA; caso contrário fica REJEITADA e não gera pontos.
- **Pontos e níveis:** ao aprovar uma ação, os pontos são somados ao usuário e o nível é recalculado: Bronze (0), Prata (200), Ouro (400), Diamante (700) e Esmeralda (1000 pontos).
- **Missões:** cada ação aprovada soma +1 de progresso nas missões ativas do usuário (não concluídas e dentro do prazo). Ao atingir o objetivo, a recompensa em pontos é creditada.
- **Conquistas:** a primeira ação aprovada do usuário desbloqueia a conquista “Primeira Ação”. Também é possível cadastrar outras conquistas.
- **Ranking:** lista os usuários da maior para a menor pontuação, sem expor o email.
- **Regras extras:** ações rejeitadas podem ser corrigidas (PUT) e reenviadas; ações aprovadas não podem ser alteradas; ao excluir uma ação aprovada, os pontos dela são retirados do usuário.

Principais regras de negócio e validações implementadas na camada Service (BO): campos obrigatórios e tamanhos máximos, email válido e não repetido, existência do usuário antes de criar ações/missões/conquistas, prazo da missão não pode estar no passado e objetivo maior que zero. Quando uma regra é violada a API devolve 400 (ou 404 se o recurso não existe) com uma mensagem em JSON.

3. Arquitetura e tecnologias utilizadas

A aplicação foi organizada em camadas, cada uma com uma responsabilidade (Separation of Concerns):

Camada / pacote	Responsabilidade	Equivalente no enunciado
model	Classes que representam as tabelas do banco (Usuario, AcaoSustentavel, Missao, Conquista, Nivel e o enum Status).	Camada modelo
repository	Persistência com JDBC: DataSource injetado, PreparedStatement e o CRUD de cada tabela.	Camada DAO (conexão por DataSource/pool)

Camada / pacote	Responsabilidade	Equivalente no enunciado
service	Regras de negócio, validações, cálculo de pontos, níveis, missões e ranking. Usa @Transactional ao gravar.	Camada BO
resource	Classes @RestController com os endpoints, verbos HTTP e status de resposta.	Camada Resource
dto e mapper	Records DTO que trafegam na API e classes Mapper que convertem DTO ↔ entidade (a entidade nunca é exposta).	—
exception	EcoRankException (regra de negócio) e RecursoNaoEncontradoException (404).	Tratamento de exceções
config	CorsFilter, que libera o acesso do front-end.	CorsFilter

Tecnologias: Java 21, Spring Boot 4.1.0 (spring-boot-starter-webmvc e spring-boot-starter-jdbc), Maven, driver Oracle ojdbc11, banco de dados Oracle, Docker e Render.

Fluxo de uma requisição: Cliente HTTP ↔ Resource ↔ Service ↔ Repository ↔ Banco de dados.

4. Tabela de endpoints (API Restful)

URL base local: http://localhost:8080 (em produção, a URL do Render). Todos os dados trafegam em JSON. Códigos usados: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found e 500 Internal Server Error.

Verbo	URI	Descrição	Corpo (JSON)	Status esperados
GET	/api/v1/usuarios	Lista todos os usuários	—	200, 500
GET	/api/v1/usuarios/{id}	Busca usuário pelo id	—	200, 404, 500
POST	/api/v1/usuarios	Cadastra usuário	nome, email	201, 400, 500
PUT	/api/v1/usuarios/{id}	Atualiza nome e email	nome, email	200, 400, 404, 500
DELETE	/api/v1/usuarios/{id}	Exclui usuário (e seus dados)	—	204, 404, 500
GET	/api/v1/acoes	Lista todas as ações sustentáveis	—	200, 500
GET	/api/v1/acoes/usuario/{idUsuario}	Lista as ações de um usuário	—	200, 404, 500
GET	/api/v1/acoes/{id}	Busca ação pelo id	—	200, 404, 500
POST	/api/v1/acoes	Registra ação: calcula pontos, valida evidência e atualiza usuário	idUsuario, categoria, descricao, linkEvidencia	201, 400, 404, 500
PUT	/api/v1/acoes/{id}	Corrige ação rejeitada e revalida	categoria, descricao, linkEvidencia	200, 400, 404, 500
DELETE	/api/v1/acoes/{id}	Exclui ação (retira os pontos se estava aprovada)	—	204, 404, 500
GET	/api/v1/missoes	Lista todas as missões	—	200, 500
GET	/api/v1/missoes/usuario/{idUsuario}	Lista as missões de um usuário	—	200, 404, 500
GET	/api/v1/missoes/{id}	Busca missão pelo id	—	200, 404, 500
POST	/api/v1/missoes	Cria missão para um usuário	idUsuario, descricao, objetivo, recompensa, prazo	201, 400, 404, 500
PUT	/api/v1/missoes/{id}	Atualiza dados da missão (progresso é automático)	descricao, objetivo, recompensa, prazo	200, 400, 404, 500
DELETE	/api/v1/missoes/{id}	Exclui missão	—	204, 404, 500
GET	/api/v1/conquistas	Lista todas as conquistas	—	200, 500
GET	/api/v1/conquistas/usuario/{idUsuario}	Lista as conquistas de um usuário	—	200, 404, 500
GET	/api/v1/conquistas/{id}	Busca conquista pelo id	—	200, 404, 500
POST	/api/v1/conquistas	Cadastra conquista para um usuário	idUsuario, nome, descricao	201, 400, 404, 500
PUT	/api/v1/conquistas/{id}	Atualiza nome e descrição	nome, descricao	200, 400, 404, 500
DELETE	/api/v1/conquistas/{id}	Exclui conquista	—	204, 404, 500
GET	/api/v1/niveis	Lista os níveis	—	200, 500
GET	/api/v1/niveis/{id}	Busca nível pelo id	—	200, 404, 500
POST	/api/v1/niveis	Cadastra nível	nome, pontosNecessarios, multiplicador	201, 400, 500
PUT	/api/v1/niveis/{id}	Atualiza nível	nome, pontosNecessarios, multiplicador	200, 400, 404, 500
DELETE	/api/v1/niveis/{id}	Exclui nível	—	204, 404, 500

Verbo	URI	Descrição	Corpo (JSON)	Status esperados
GET	/api/v1/ranking	Ranking de usuários por pontuação (posicao, nome, pontos, nivel)	—	200, 500

Observações: usar um verbo não permitido em uma URI retorna 405 (Method Not Allowed) e uma URI inexistente retorna 404, ambos tratados pelo próprio Spring. Os campos id, status, pontos, data, progresso e nível são definidos pelo sistema e ignorados quando enviados no corpo.

5. Exemplos de requisição e resposta

1) Cadastrar usuário – POST /api/v1/usuarios

```
Corpo:
{ "nome": "Maria Silva", "email": "maria@email.com" }

Resposta 201 Created:
{ "id": 1, "nome": "Maria Silva", "email": "maria@email.com",
  "pontos": 0, "nivel": "Bronze", "recompensa": "" }
```

2) Registrar ação sustentável – POST /api/v1/acoes

```
Corpo:
{ "idUsuario": 1, "categoria": "Plantio",
  "descricao": "Plantei uma muda de ipê no bairro",
  "linkEvidencia": "https://exemplo.com/foto-ipe.jpg" }

Resposta 201 Created:
{ "id": 1, "idUsuario": 1, "categoria": "Plantio",
  "descricao": "Plantei uma muda de ipê no bairro",
  "linkEvidencia": "https://exemplo.com/foto-ipe.jpg",
  "status": "APROVADA", "pontos": 70, "data": "2026-10-29" }
```

3) Ranking – GET /api/v1/ranking

```
Resposta 200 OK:
[ { "posicao": 1, "nome": "Maria Silva", "pontos": 70, "nivel": "Bronze" } ]
```

4) Erro de validação – POST /api/v1/usuarios com email repetido

```
Resposta 400 Bad Request:
{ "mensagem": "Já existe um usuário cadastrado com este email" }
```

Teste com cURL:

```
curl -X POST http://localhost:8080/api/v1/usuarios \
  -H "Content-Type: application/json" \
  -d '{"nome":"Maria Silva","email":"maria@email.com"}'

curl -X GET http://localhost:8080/api/v1/ranking
```

6. Protótipo – prints das telas implementadas

Como o EcoRank é uma API, as telas abaixo mostram o sistema em funcionamento: a aplicação iniciada, as requisições testadas no Postman, o banco de dados e a aplicação publicada na nuvem. Cada print vem acompanhado de uma explicação resumida.

Print 1 – Aplicação iniciada no IntelliJ

[COLAR AQUI O PRINT]
Console do IntelliJ mostrando "Tomcat started on port 8080" e "Started EcorankApplication".

Explicação: Ao executar o método main da classe EcorankApplication o Spring Boot sobe o Tomcat embutido na porta 8080 e conecta ao Oracle pelo DataSource.

Print 2 – Cadastro de usuário (POST /api/v1/usuarios)

[COLAR AQUI O PRINT]
Postman com o corpo JSON e a resposta 201 Created.

Explicação: O usuário é criado com 0 pontos e nível Bronze, e a API devolve o DTO com o id gerado pelo banco.

Print 3 – Registro de ação sustentável (POST /api/v1/acoes)

[COLAR AQUI O PRINT]
Postman com a resposta 201 e status APROVADA.

Explicação: O Service calcula os pontos pela categoria, valida o link de evidência e aprova a ação.

Print 4 – Usuário atualizado (GET /api/v1/usuarios/{id})

[COLAR AQUI O PRINT]
Postman mostrando pontos e nível atualizados após a ação.

Explicação: Após a aprovação, os pontos foram somados e o nível recalculado automaticamente.

Print 5 – Ranking (GET /api/v1/ranking)

[COLAR AQUI O PRINT]
Postman ou navegador com a lista ordenada.

Explicação: Usuários ordenados do maior para o menor número de pontos; o email não é exibido.

Print 6 – Tratamento de erro (400 ou 404)

[COLAR AQUI O PRINT]
Postman com uma resposta de erro e a mensagem em JSON.

Explicação: Regras violadas retornam o código HTTP adequado e uma mensagem clara, sem expor detalhes internos.

Print 7 – Banco de dados no SQL Developer

[COLAR AQUI O PRINT]
Tabelas USUARIO, ACAO_SUSTENTAVEL, MISSAO, CONQUISTA e NIVEL com dados.

Explicação: Confirma que os dados enviados pela API foram gravados no Oracle.

Print 8 – Deploy no Render e teste pela URL pública

[COLAR AQUI O PRINT]

Painel do Render com status "Live" e a URL https://....onrender.com/api/v1/ranking aberta no navegador.

Explicação: A API está publicada na nuvem e respondendo.

Print 9 – Integração com o front-end (se houver)

[COLAR AQUI O PRINT]

Tela do front-end listando dados vindos da API.

Explicação: Mostra o front-end consumindo os endpoints, o que só funciona graças ao CorsFilter.

Atenção: substituir cada quadro amarelo por uma captura de tela real antes de exportar o PDF final.

7. Modelo de Entidade-Relacionamento (MER)

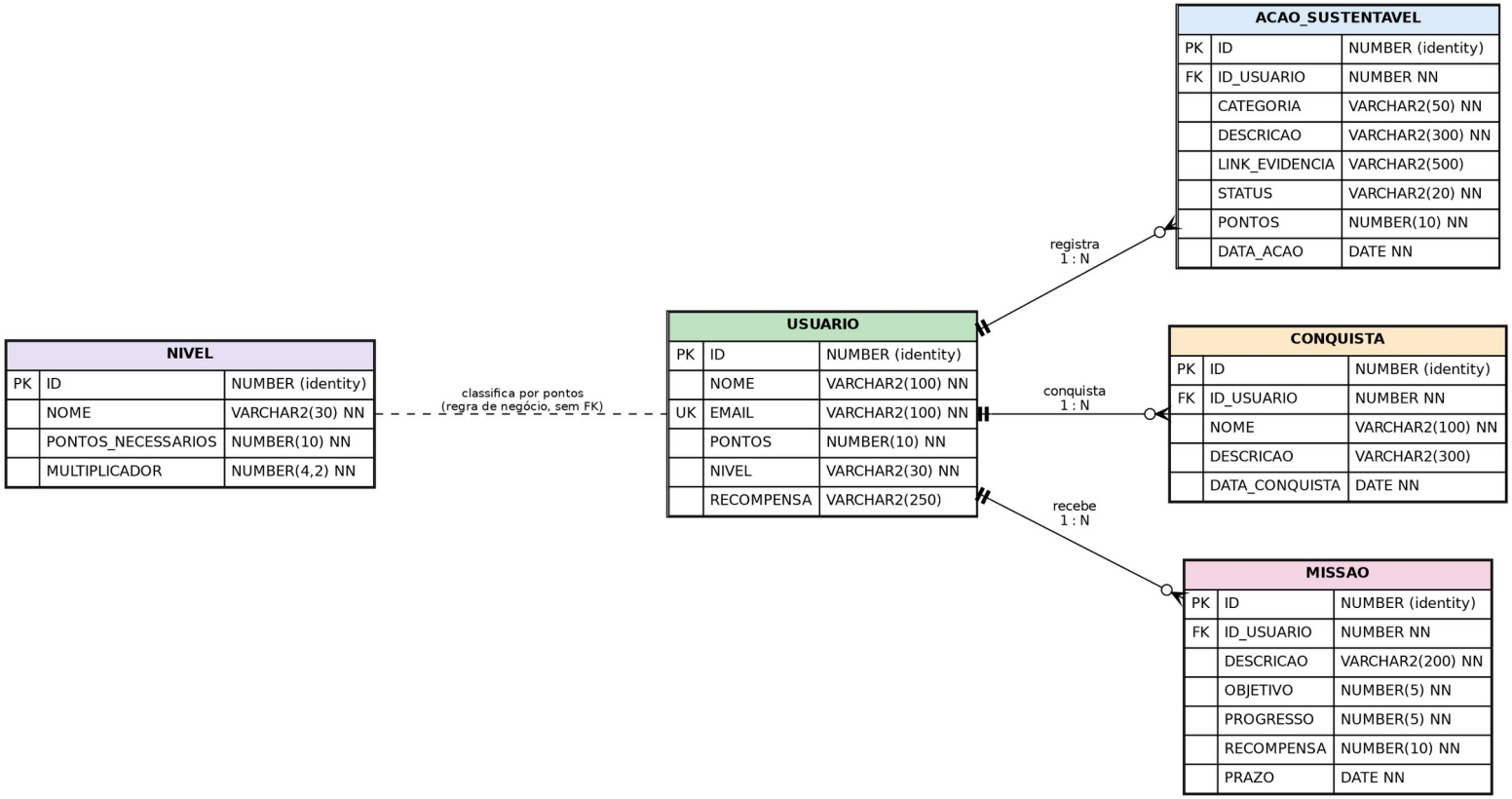


Figura 1 – MER do EcoRank (PK = chave primária, FK = chave estrangeira, UK = única, NN = não nulo)

Relacionamentos: um USUARIO possui várias AÇÕES_SUSTENTAVEIS, várias CONQUISTAS e várias MISSOES (1:N, com chave estrangeira ID_USUARIO e exclusão em cascata). A tabela NIVEL é uma tabela de apoio: o nível do usuário é definido pelos pontos, sem chave estrangeira. As chaves primárias são geradas automaticamente pelo Oracle (identity). O script completo está em script_banco.sql.

8. Diagrama de classes atualizado

Os diagramas representam o projeto Java entregue (pacote br.com.ecorank.ecorank). Foram divididos por camada para ficarem legíveis. Legenda: seta fechada vazia = herança; tracejada com ponta vazia = implementação; seta aberta = associação/dependência; tracejada = uso (injeção de dependência).

8.1 Camada Model (entidades)

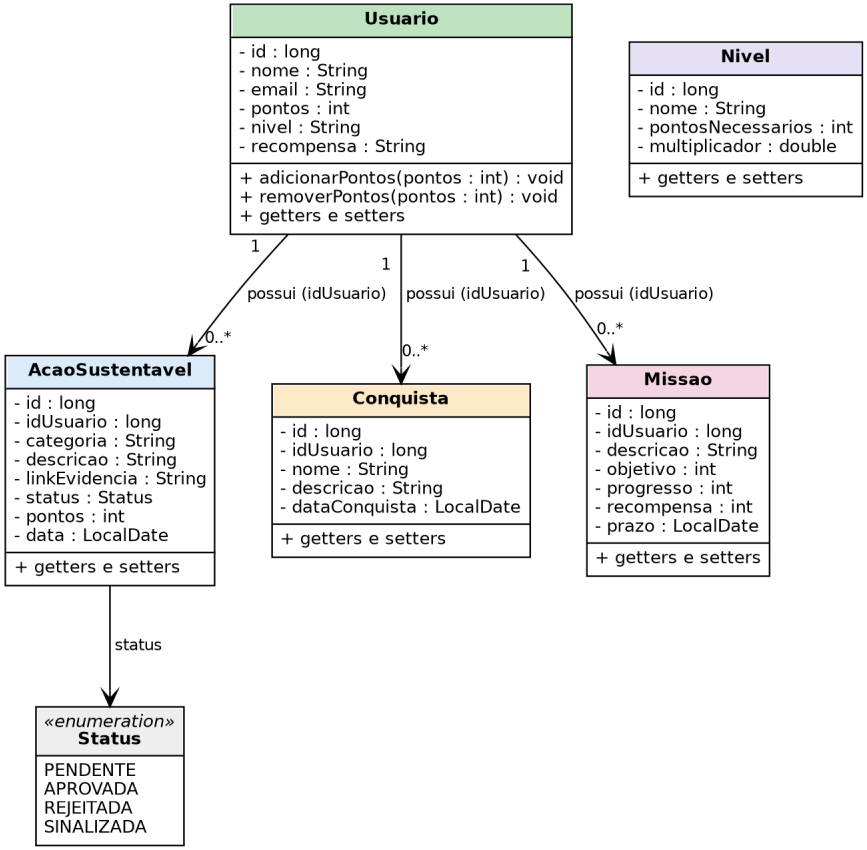


Figura 2 – Classes do pacote model e associações (Usuário 1 → 0.. Ações, Conquistas e Missões)*

8.2 Camada Service (BO)

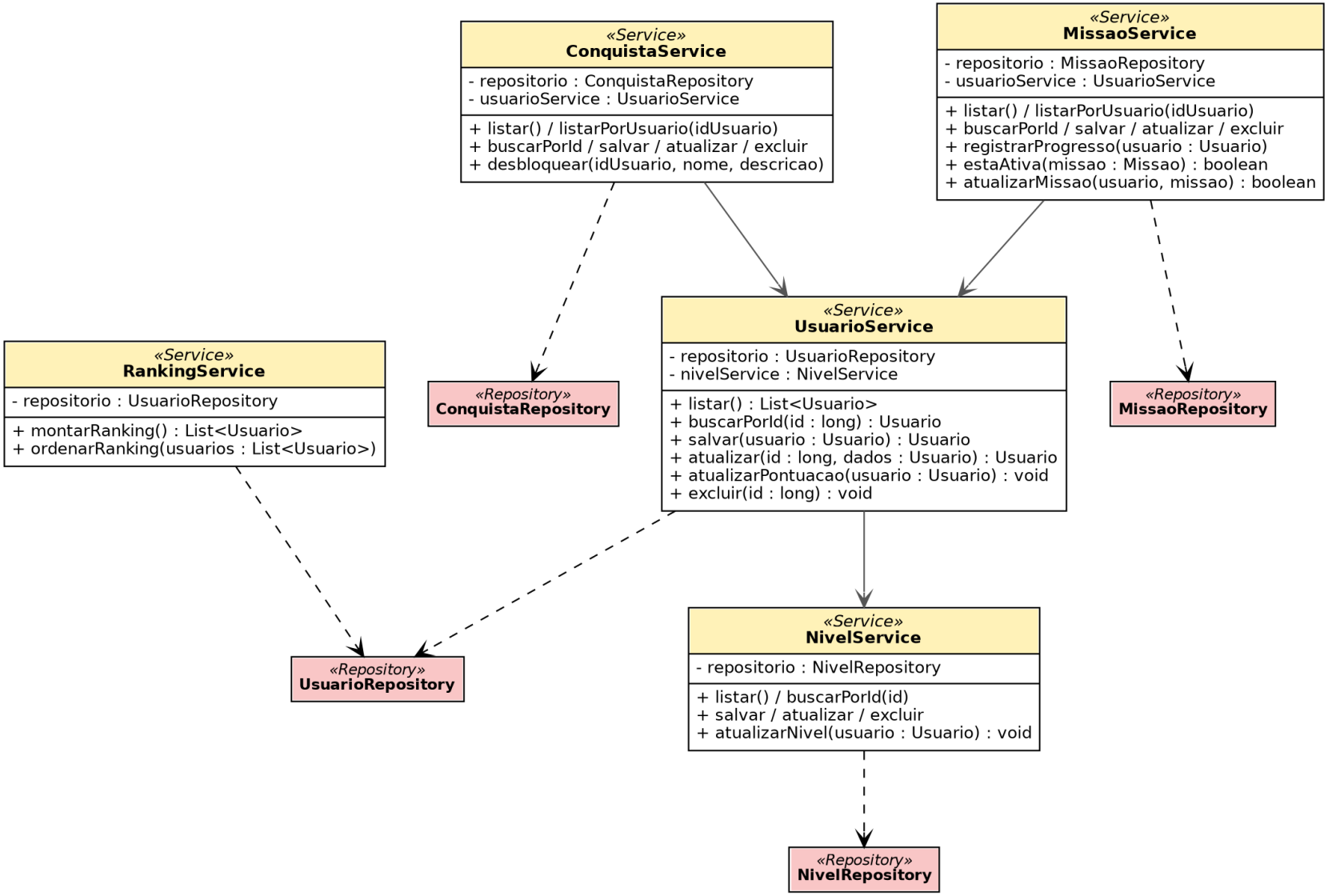


Figura 3 – Services de usuário, nível, ranking, conquista e missão

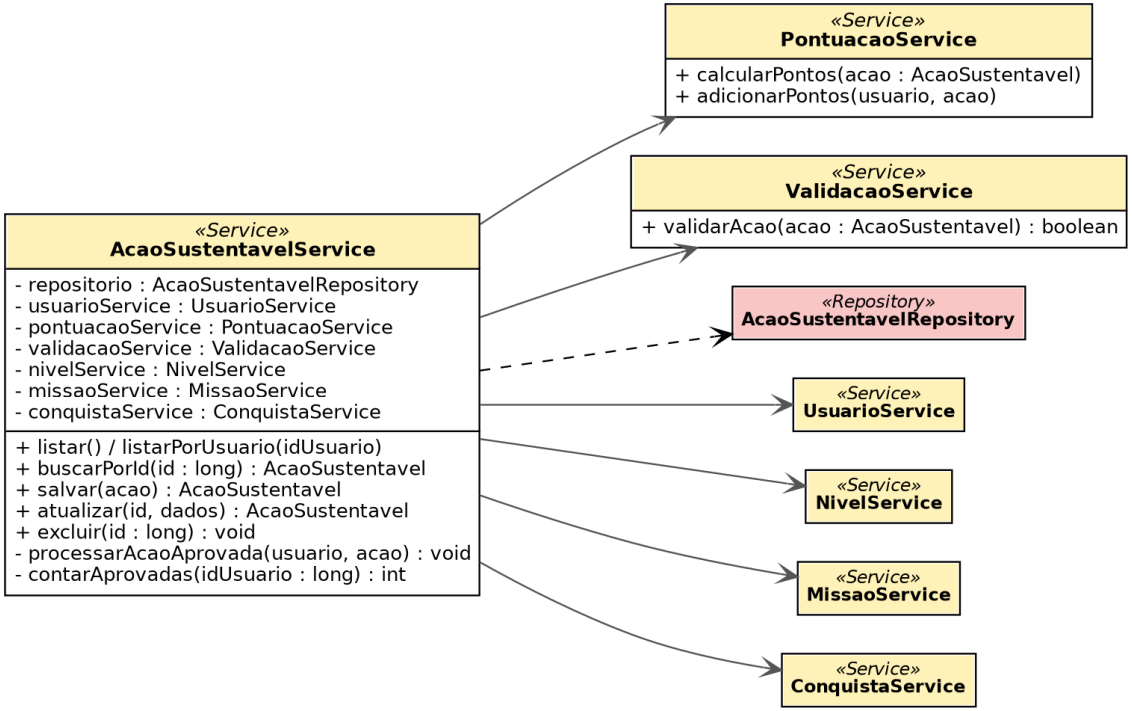


Figura 4 – AcaoSustentavelService, que orquestra as demais regras de negócio

8.3 Camada Repository (DAO)

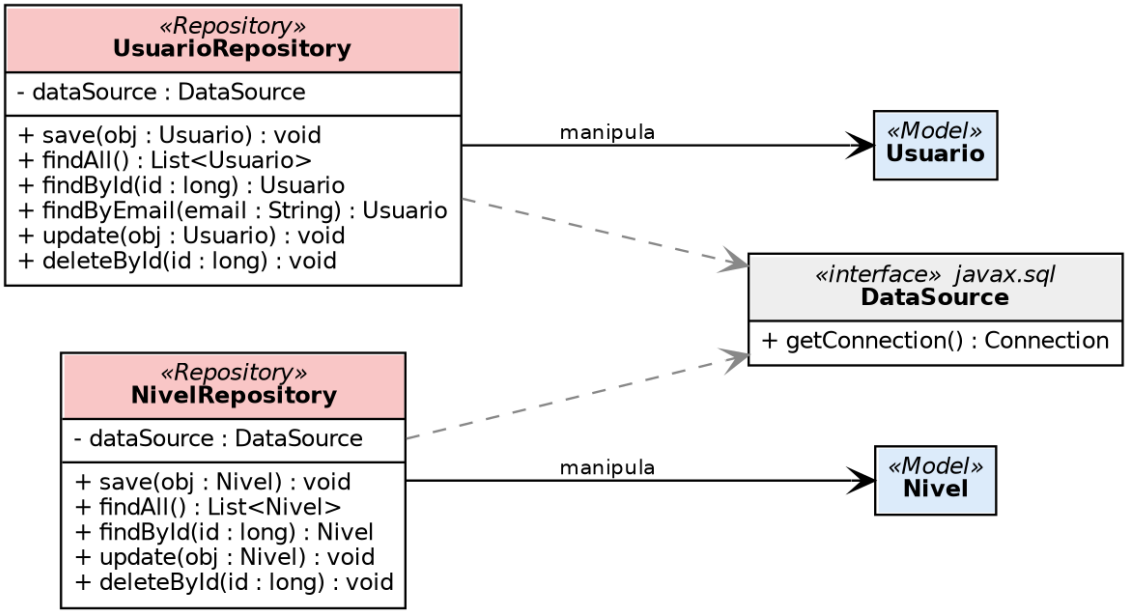


Figura 5 – Repositories de usuário e nível (uso do DataSource)

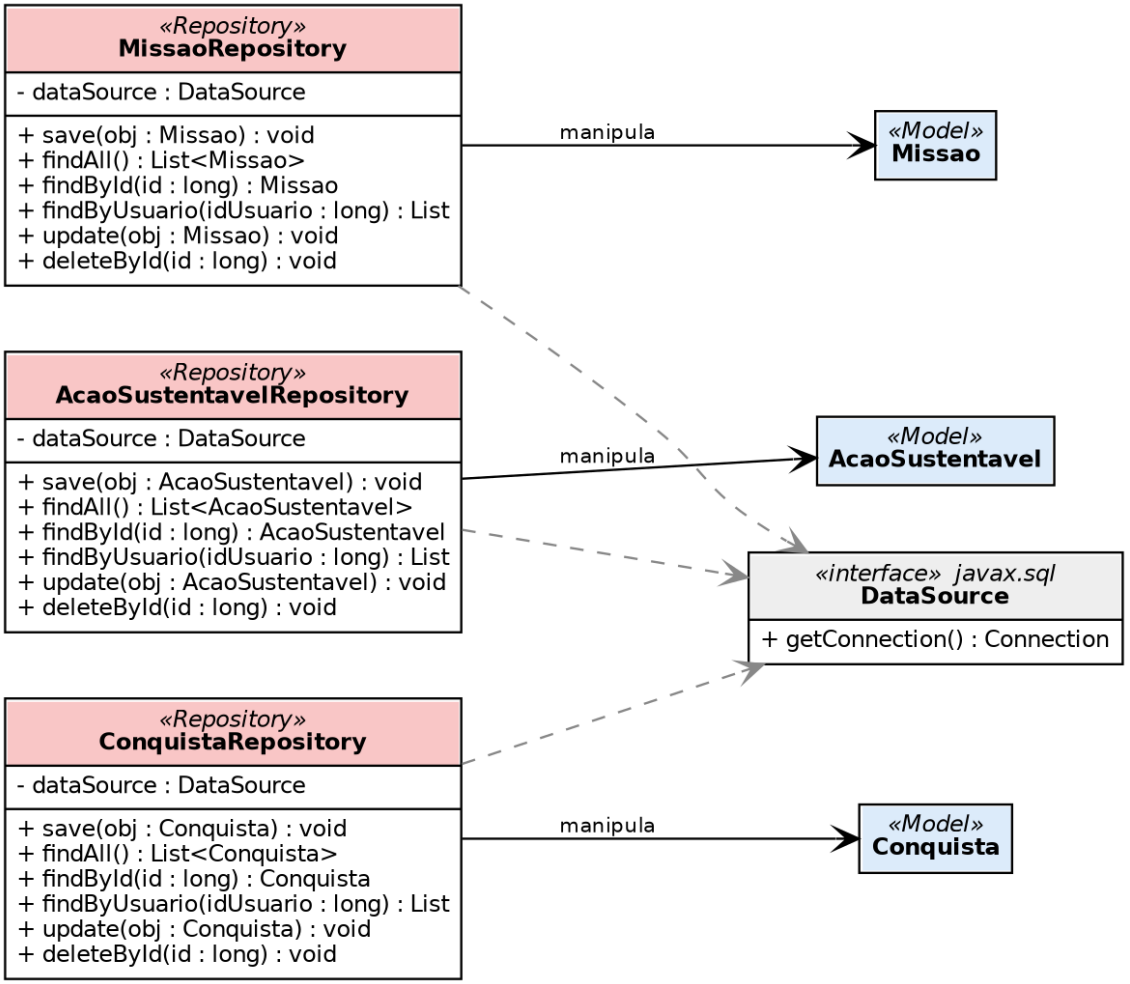


Figura 6 – Repositories de ação sustentável, missão e conquista

8.4 Camada Resource, DTO e Mapper

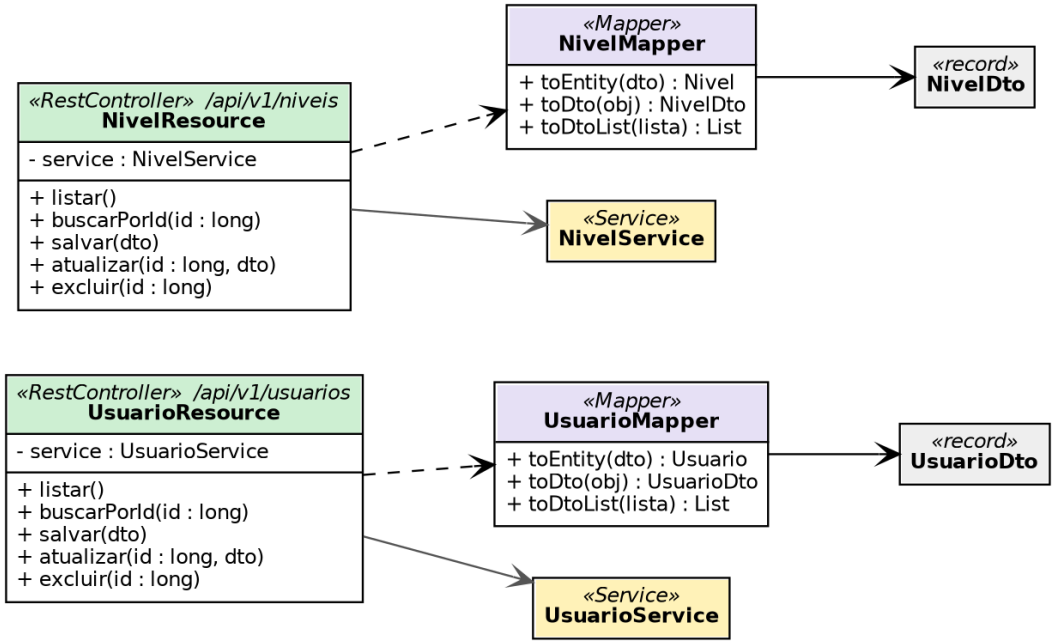


Figura 7 – Resources de usuário e nível

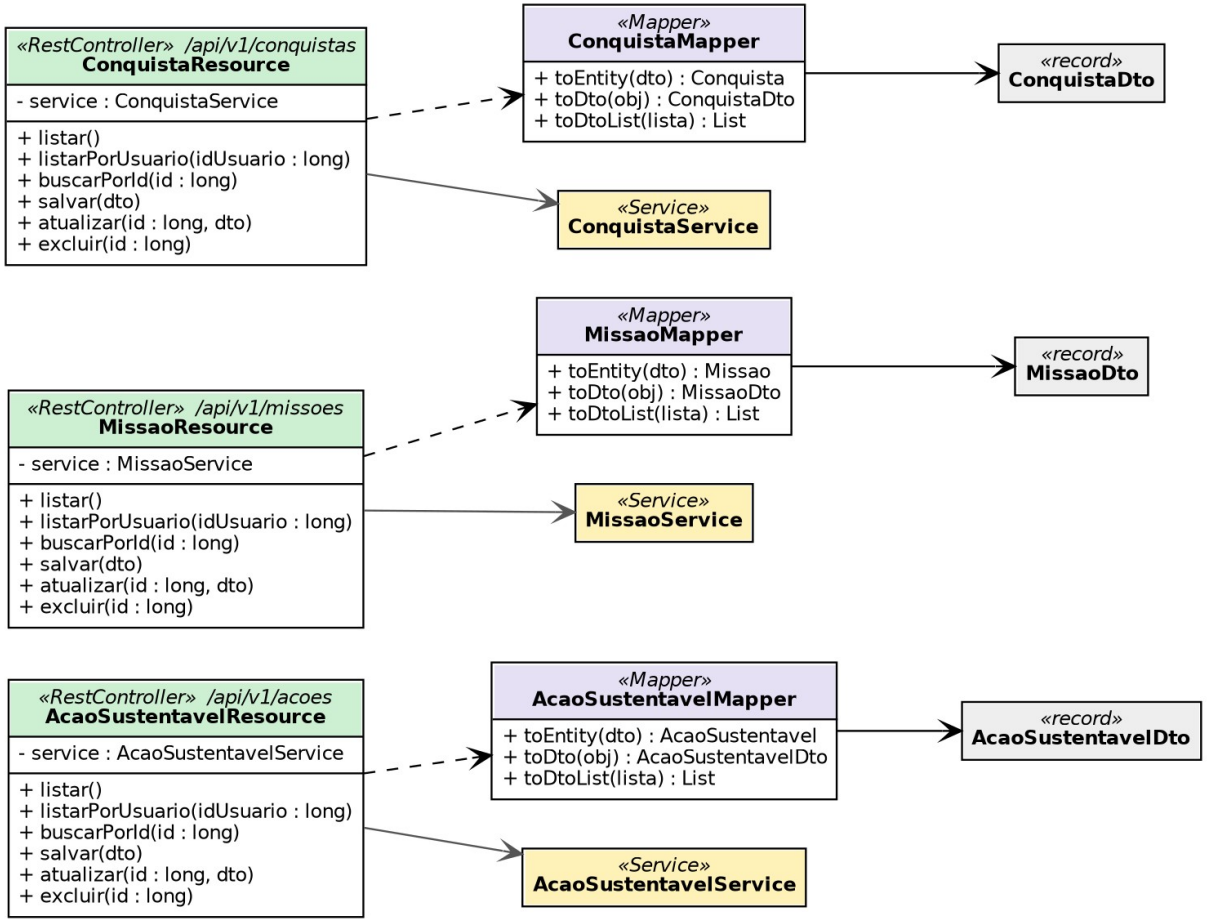


Figura 8 – Resources de ação sustentável, missão e conquista

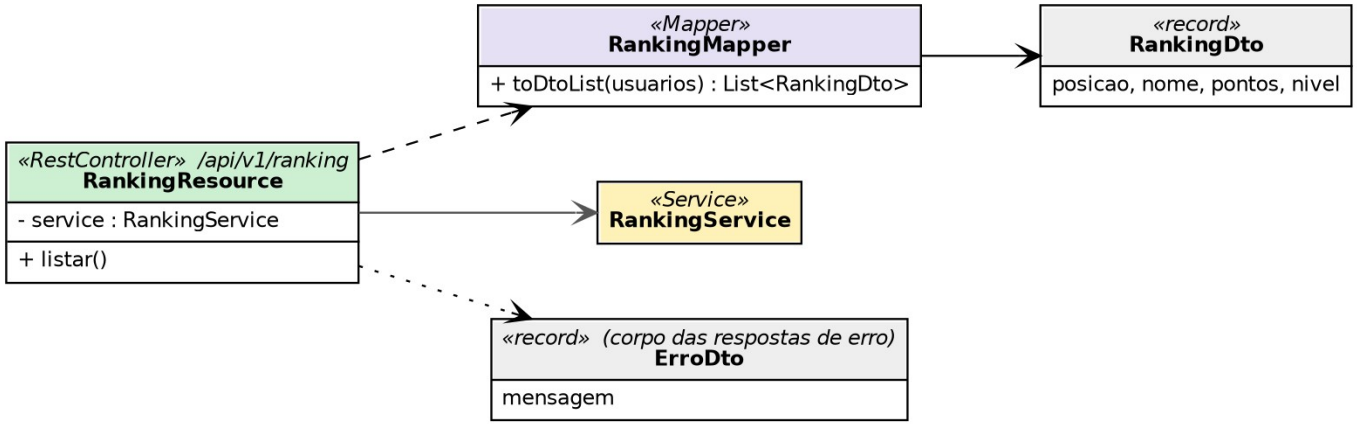


Figura 9 – RankingResource e ErroDto

8.5 Herança e realização (exceções e CorsFilter)

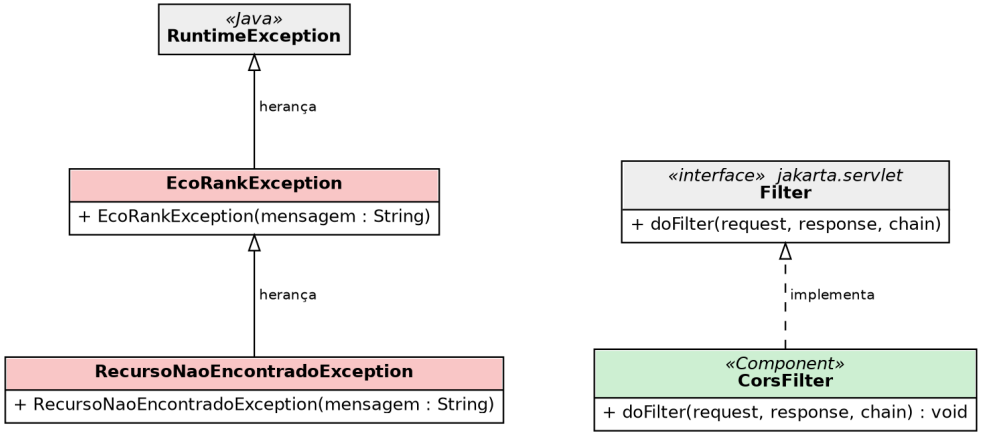


Figura 10 – Hierarquia das exceções e CorsFilter implementando Filter

9. Procedimentos para rodar a aplicação

9.1 Ferramentas e versões necessárias

Ferramenta	Versão / observação
Java JDK	21 (ou superior)
IDE	IntelliJ IDEA (também funciona no Eclipse, NetBeans ou VS Code)
Maven	3.9+ (já vem embutido no IntelliJ)
Spring Boot	4.1.0 (definido no pom.xml)
Banco de dados	Oracle (o mesmo da FIAP: oracle.fiap.com.br:1521:orcl) e SQL Developer para rodar o script
Teste da API	Postman, cURL ou o navegador (para os GET)
Git / GitHub	Para versionar e publicar o código
Docker (opcional)	Só para testar a imagem localmente; no Render o build é feito pela própria plataforma

9.2 Como importar o projeto

1. Descompacte o arquivo EcoRank.zip.
2. No IntelliJ, escolha File > Open, selecione a pasta EcoRank (ou o arquivo pom.xml) e clique em “Open as Project”.
3. Aguarde o Maven baixar as dependências (barra de progresso no canto inferior direito).
4. Confira em File > Project Structure se o SDK do projeto é o Java 21.

9.3 Passo a passo da execução

5. Abra o SQL Developer, conecte no Oracle e execute todo o conteúdo do arquivo script_banco.sql (cria as tabelas e insere os níveis iniciais).
6. Abra src/main/resources/application.properties e confira a URL, o usuário e a senha do banco. Os valores podem ser trocados pelas variáveis de ambiente DB_URL, DB_USER e DB_PASSWORD.
7. Abra a classe EcorankApplication e clique em Run (ou execute o método main). O console deve mostrar “Tomcat started on port 8080”.
8. Teste no navegador ou no Postman: http://localhost:8080/api/v1/ranking (deve retornar uma lista, vazia no início).
9. Cadastre um usuário (POST /api/v1/usuarios) e uma ação (POST /api/v1/acoes) seguindo os exemplos da seção 5.

Alternativa pelo terminal (na pasta do projeto):

```
mvn clean package -DskipTests
java -jar target/ecorank-0.0.1-SNAPSHOT.jar
```

Problemas comuns: erro ORA-00942 significa que o script SQL ainda não foi executado; erro de conexão significa URL, usuário ou senha incorretos (ou fora da rede da FIAP/VPN); porta 8080 ocupada pode ser trocada com a variável PORT.

10. Deploy da aplicação na nuvem (Render)

O Render não possui ambiente nativo para Java, por isso a aplicação é publicada com Docker. O arquivo Dockerfile fica na raiz do projeto: na primeira etapa o Maven gera o .jar (mvn clean package -DskipTests) e na segunda etapa uma imagem leve do Java 21 executa a aplicação.

```
FROM maven:3.9-eclipse-temurin-21 AS build
```

```
WORKDIR /build
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /build/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Passo a passo:

10. Suba o projeto para um repositório no GitHub (git add, git commit e git push).
11. Acesse render.com e entre com a conta do GitHub.
12. Clique em New > Web Service e conecte o repositório do projeto.
13. Em Language escolha Docker, Branch main e deixe o Dockerfile Path como ./Dockerfile.
14. Em Environment Variables cadastre DB_URL, DB_USER e DB_PASSWORD com os dados do Oracle.
15. Escolha a instância gratuita (Free) e clique em Create Web Service.
16. Quando o status ficar “Live”, teste: https://[nome-do-servico].onrender.com/api/v1/ranking.

Observação: na instância gratuita o serviço “dorme” após alguns minutos sem uso, então a primeira requisição pode demorar cerca de um minuto.

11. Links da entrega

Item	Link
Repositório no GitHub	[colar aqui o link do repositório]
Deploy na nuvem (Render)	[colar aqui o link do deploy]